

梅花林自动拾取KFS

路径规划部分

可取桩位为

左一列：1, 4, 7, 10

中一列：2, 11

右一列：3, 6, 9, 12

以下提到的速度均为世界坐标系下的速度；采用右手系

至于梅花林地图的简化处理。

将原本\$34\$的梅花林加上外围一圈的通道切割成\$56\$的一个地图(梅花林是在中间\$3*4\$的矩形方格，外围一圈是通道)

方格的数字号按从左到右依序排列上去。

每个方格都为\$1200\text{mm} * 1200\text{mm}\$

则原本梅花桩数字号对应新的排列方式的关系是(伪代码)

```
static inline int8_t MFNum_TransforMapNum(int8_t MFNum)
{
    if(MFNum < 1 || MFNum > 12 || (MFNum == 5 || MFNum == 8))
        return -1;

    return MFNum + 6 + 2 * ((MFNum - 1) / 3.0);
}

static inline int8_t MapNum_TransformMFNum(int8_t mapNum)
{
    int8_t MFNum_ = mapNum - 6 - 2 * ((mapNum - 7) / 3);

    if(MFNum_ < 1 || MFNum_ > 12 || (MFNum_ == 5 || MFNum_ == 8))
        return -1;

    return MFNum_;
}
```

那么对应的，要拾取每个梅花桩上的KFS，底盘沿通道行进，在到达目标桩跟前的通道方块A之前；要先在A之前的通道方块B开启拾取预测计算以及最少在B此处将非沿着通道方向的速度降低为0(即此处底盘走的通道方向为Y轴方向的话，那么X轴向速度就要为0)；

如果不考虑最优路径的话，对应MFNum编号的梅花桩的拾取通道可以有多解(最多3解)，满足关系(伪代码)

```
typedef struct{
    int8_t result1 = 0;
```

```
    int8_t result2 = 0;
    int8_t result3 = 0;
}RoadResult_S;
static inline RoadResult_S MFNum_ToRoadResult(int8_t MFNum)
{
    RoadResult_S result = {0, 0, 0};
    if(MFNum < 1 || MFNum > 12 || (MFNum == 5 || MFNum == 8))
        return result;

    int8_t mapNum_ = MFNum_Transform(MFNum);

    int8_t candidate[4];
    candidate[0] = mapNum_ - 6;
    candidate[1] = mapNum_ - 4;
    candidate[2] = mapNum_ + 4;
    candidate[3] = mapNum_ + 6;

    for(int i = 0; i < 4; i++)
    {
        int8_t mf = MapNum_TransformMFNum(candidate[i]);
        //过滤梅花桩块
        if(mf < 1 || mf > 12 || mf == 5 || mf == 8)
            candidate[i] = 0;
    }

    int8_t validResults[3] = {0};
    int validCount = 0;
    //过滤0
    for(int i = 0; i < 4 && validCount < 3; i++)
    {
        if(candidate[i] != 0)
        {
            validResults[validCount++] = candidate[i];
        }
    }

    //排列
    for(int i = 0; i < validCount - 1; i++)
    {
        for(int j = 0; j < validCount - 1 - i; j++)
        {
            if(validResults[j] > validResults[j+1])
            {
                int8_t temp = validResults[j];
                validResults[j] = validResults[j+1];
                validResults[j+1] = temp;
            }
        }
    }

    result.result1 = validResults[0];
    result.result2 = validResults[1];
    result.result3 = validResults[2];
}
```

```
    return result;
}
```

求解方格的行列坐标，关系式（伪代码）

```
// 2D点结构体

typedef struct {
    float x, y;
} Point2D;

static inline Point2D MapNum_ToMatrixPos(int8_t MapNum)
{
    Point2D result_ = {0, 0};

    result.y = (MapNum - 1) / 5 + 1;
    result.x = (MapNum - 1) % 5 + 1;
    return result_;
}
```

提前给30个方格预设好真实坐标

```
const Point2D MapNum_RealPos[30] = {...};

Point2D TargetMapNum[2] = {..., ...};
```

然后通过先前的函数`MFNum_ToRoadResult`和`MapNum_ToMatrixPos`查找到目标两个梅花桩拾取时候需经过的前一桩的行列坐标的多解；

并通过当前底盘的真实位置，去计算经过的前一桩的优解(必经过的点)，然后先经过优解点1，再经过优解点2，最后经过而所有路径的终点都是19号格子(上斜坡到竞技场)。

qizhon

伪代码↓

```
typedef struct {
    int8_t targetMapNum1;
    int8_t targetMapNum2;
    const int8_t endMapNum = 19;
}PathNode;

static inline PathNode FindBestNode(int8_t target_MFNum, Point2D robotPos_now)
```

```
{  
  
}
```

机械臂自动吸取KFS部分(现有平地的方案)

在底盘中心到达最优解点的中心时候，开始计算吸取时机。